

**The Unreasonable Extensibility of Computation:
What a Fixed Machine Can Do, and When It Is Possible to
Know**

PIATRA . INSTITUTE

September 2026

Abstract

A handheld games console from 2004 now generates text locally. A published runtime puts a 90-million-parameter hybrid attention and state-space model on a Sony PSP with 64 MB of memory and a 333 MHz processor, at about 0.55 tokens per second, with the weights on a memory card and no network. The processor did not change. What changed is the stock of representations, algorithms, runtimes, and trained parameters that can be brought to it. This paper takes that observation and computes with it. Capability is a relation among a machine, an artifact, a task, a budget, and a quality threshold, and the set of tasks a fixed machine can perform is therefore indexed by a date. Three results follow. The first is arithmetic on the byte format the runtime publishes, with one free parameter solved from the one throughput figure it reports, which leaves the rest as prediction: full-precision weights need 7.24 times the console’s weight cache, four-bit block quantisation reduces that by 7.11 and still leaves the model at 1.02 of the cache, so the residual 0.87 MB per token must come off the card, and the capability is a conjunction. Of the sixteen subsets of the four techniques the runtime uses, 2 reach a 0.4 tokens-per-second floor, three techniques are individually necessary, the fourth buys speed rather than feasibility, and on the 32 MB revision of the same console 0 of 16 subsets run at all. Compression also relocates the constraint rather than removing it: at or above 6 bits the system is storage-bound and at or below 5 bits it is arithmetic-bound, so from the format in use a perfect representation would buy 1.06 times more speed and nothing else. The second result concerns why continuous improvement looks like discontinuous capability. Calibrating a family-improvement distribution to four published summary statistics of a 113-family census and putting a multidimensional budget over it, between 0.83 and 0.89 of all improvement events change no capability predicate at all, and the burstiness of capability crossings relative to the burstiness of the efficiency gains that cause them rises from 0.84 to 1.67 as the budget binds harder, with the concentration of crossings in the top decile of families rising from 0.24 to 0.41 alongside it. Amplification is real and it is conditional, and where the budget barely binds there is none. In the most demanding regime the fixed device with current software overtakes a device with 10 times its budget running frozen software in year 25. The third result is the one that cuts against the first two. Known capability accumulates; deployable capability requires every component of a stack to remain runnable at once, so it saturates rather than growing, at 0.43 of known for a five-component stack against 0.83 for a single component, and it turns over and falls to 0.39 of its peak when interest wanes. For a five-deep stack to survive as long as the mean interval between improvements in its own field, each component would have to last 201 years. The machine is the easy part to keep.

A console from 2004, generating text

The artifact is specific, and its documentation is unusually precise about its own limits. A custom runtime executes Falcon-H1-Tiny-90M-Instruct, a 90-million-parameter instruction-tuned model whose 24 layers each run grouped-query attention and a Mamba2 recurrent branch in parallel, on the PSP’s 333 MHz MIPS processor, generating about 0.5 to 0.6 tokens per second on a PSP-3000 (Thatblend, 2026; Technology Innovation Institute, 2026; Gu and Dao, 2023). The weights sit on

the memory card. Nothing is streamed from a server. Every property of the artifact stated here comes from the runtime’s own documentation and the model card, and the console’s release dates are scene-setting rather than a premise of anything computed below.

The runtime states what it needs. Supported hardware is the PSP-2000, PSP-3000, Street, or Go with custom firmware; “a PSP-1000 only has 32 MB of RAM and it won’t work”. Dense weights are stored in a block format of 32 weights and one 16-bit scale per 18 bytes, with rows independently addressable so that a single embedding row can be sought without parsing. Norms, convolution weights, and the state-space tensors stay at full precision, about 0.5 MB in total. The tokenizer’s 32,768 pieces are embedded in the file. The whole file is 52 MB, which “doesn’t fit in a 64 MB console alongside ~8 MB of runtime state”, so the runtime allocates correctness-critical state first, caches 44 MB of heap plus a 4 MB volatile partition the firmware is not using, and “streams the last few MB from the memory stick per token”. The key-value cache is 8-bit with a scale per head. The matrix-vector kernels run on the vector unit rather than the scalar floating-point unit. And the author is candid about the output: the model “can answer basic questions”, writes poems and short stories, produces code that does not work, and is “not really useful for anything”.

Take that last sentence seriously, because the interesting claim does not depend on the model being good. It depends on the model running at all.

The question the artifact raises is not whether the console was always capable of this. In one sense it obviously was, since the configuration of bits that constitutes the running system was always a physically reachable configuration of that hardware. The question is what it means to have said, in 2004, what that machine could do. Whatever was meant by such a statement then, it was a statement about what anyone then knew how to make the machine do within a budget. It cannot have been a statement about the set of physically reachable configurations, since nobody could enumerate that set and no procedure can. That statement was true when made and is false now, and the machine did not change.

Four senses of what a machine can do

The confusion in the vicinity is worth separating carefully, because three of the four available senses are not the interesting one.

Physical possibility is the set of state transitions the device supports. Loading software changes which configuration the memory is in, so in a strict sense the hardware is not unchanged between a console running a game and the same console running an inference loop. Throughout this paper *fixed hardware* means the architecture, clock ceiling, memory and storage capacities, bandwidths, peripherals, and power envelope are held constant while the information loaded into them varies. The set of physically reachable configurations does not move.

Computability in principle is what Turing settled. A universal machine can simulate any machine given its description (Turing, 1937), which is why a games console and an inference engine can be the same object. This is the sense in which the answer to the question was fixed in 2004, and it is

not useful, because it is budget-blind. Nothing in universality says an implementation fits in 48 MB, returns a token in under two seconds, or stays inside a battery.

Resource-bounded feasibility is the sense that moves. The distinction was drawn by Hartmanis and Stearns (1965) when they indexed computability by the resources a machine spends, and it is where the philosophical content sits, since almost every question about what a computer can do for us is a question about a budget rather than about a limit (Aaronson, 2013).

Operational capability adds a quality threshold, because a task performed badly enough is not the task. The console runtime supplies its own illustration of why these must be tracked separately. Its documentation notes that cache size “only affects speed, greedy output is identical at any setting”: the same artifact moves along the resource axis without moving along the quality axis at all.

Write a task as an input space, an output space, a distribution over inputs, and a performance measure. Write H for the fixed hardware, p for an executable artifact construed broadly enough to include code, trained parameters, tables, indexes, and the runtime, $Q(p, H, \tau)$ for task quality and $R(p, H, \tau)$ for a vector of resources spent, and B for a budget vector. Let K_t be the artifacts that exist and can be obtained at time t . The feasible capability set is

$$\mathcal{C}_t(H, B, q) = \{ \tau : \exists p \in K_t, Q(p, H, \tau) \geq q, R(p, H, \tau) \preceq B \},$$

and the substrate’s intrinsic set $\mathcal{C}^*$ replaces K_t with every artifact instantiable on H at all. Then $\mathcal{C}_t \subseteq \mathcal{C}^*$, the right-hand side is fixed, and the left-hand side is dated.

The gap between them is uncomputable, which is a stronger condition than unknown. The shortest program for a task is not computable (Li and Vitányi, 2019), nontrivial semantic properties of programs are undecidable (Rice, 1953), and Blum’s speed-up theorem gives computable functions for which every algorithm has an asymptotically faster one, so that for those functions there is no best program to find (Blum, 1967). Hutter (2002) exhibits an algorithm asymptotically as fast as the fastest algorithm for any well-defined problem, with a constant that makes it useless, which is as clean a demonstration as exists that the in-principle answer and the budgeted answer are different objects. A machine’s intrinsic feasible set is not a list anyone was ever going to finish.

Three frontiers rather than one are needed, because knowing is not having:

$$\mathcal{C}_t^{\text{deployed}} \subseteq \mathcal{C}_t^{\text{accessible}} \subseteq \mathcal{C}_t^{\text{known}} \subseteq \mathcal{C}^*.$$

An implementation has been described; the code, toolchain, parameters, and expertise to run it exist and can be got; the thing actually runs on the device. The third section of this paper is about the first inclusion, which is the one that can reverse.

The closest precursor is Moor (1985), who called computers logically malleable: they can be shaped to any activity characterisable through inputs, outputs, and connecting logical operations. That is the structural property. What follows here is the historical trajectory of the reachable region under a

budget, which malleability permits and does not describe. The objection that any physical process can be read as computing anything is answered by the mechanistic account: a system computes in virtue of its causal organisation rather than in virtue of an interpretation someone assigns it (Piccinini, 2007), and the taxonomy of what makes something a computer in the first place turns on how its states are manipulated rather than on what anyone says about them (Piccinini, 2008). The console is answering unseen prompts. No reinterpretation of an idling processor does that.

What the borrowed title does and does not claim

Wigner (1960) was puzzled by the applicability of mathematics developed for its own reasons to a physical world that had no obligation to obey it. The formula has been borrowed once already for computation, by Halevy, Norvig and Pereira (2009), who argued that behaviour worth having could come out of data rather than out of a designed model, which is the ancestor of the trained parameters the console runs.

The disanalogy has to be stated rather than glossed over, because it is what makes the present case weaker than Wigner’s and more tractable. Wigner’s mathematics was mostly developed without the physical application in view, so the fit is genuinely unexplained. Algorithms and representations are designed for computational substrates, so the fit between a quantisation scheme and a memory budget is not a surprise of the same kind. Nothing about extensibility is mysterious: programmability predicts it, and Moor said so forty years ago.

Three things about it do want explaining, and this paper’s three results address them in turn. The magnitude, since a factor of 7.11 in representation and a factor of 3.2 in kernels are not small. The temporal reach, since the accumulated stock arrives two decades after the hardware. And the semantic distance, since the resulting description of the object belongs to a different category of thing than the one it was sold as. *Unreasonable* here means unexpectedly large rather than mathematically unbounded, and the finite-state objection is conceded in full: a console with finite storage has a finite set of reachable configurations, and nothing here claims otherwise.

The technique lattice

Everything in this section is arithmetic on the byte format the runtime publishes, with a single free parameter. The scalar arithmetic rate is solved rather than assumed, so that the complete four-technique stack reproduces the throughput the runtime reports, which fixes it at 33.3 million multiply-accumulate pairs per second and leaves every other row below a prediction rather than a fit. The storage bandwidth is stipulated at 8 MB per second and swept.

The four techniques each have a literature. Reduced-precision integer representation with per-block scales is the standard mechanism for fitting a network into a smaller budget without retraining it (Jacob et al., 2018; Frantar and Alistarh, 2022). Hand-written kernels for a specific vector unit are the same activity that superoptimisation automated four decades ago, and the point of that work was precisely that the lowest layer of the stack remains an object of search rather than a solved problem (Massalin, 1987). Streaming and caching are system-boundary engineering, which

changes what counts as resident without changing the hardware. What the arithmetic below adds is which of them the capability actually rests on.

The block format stores 32 weights and one 16-bit scale in 18 bytes, which is 0.5625 bytes per weight against 4 for full precision, a factor of 7.11. Applied to 91.1 million parameters the four representations give 347.5 MB at full precision, 173.8 at half, 92.3 at 8 bits with the same block scale, and 48.87 in the format actually used. Against the 48 MB the runtime can cache, those are 7.24, 3.62, 1.92, and 1.02 of budget. Not one of them fits.

That is the first thing the arithmetic says, and it is not what the compression story leads one to expect. Four-bit quantisation does not make the model fit. It moves the model from 7 times too large to 2 percent too large, and the last 2 percent has to be bought somewhere else. Adding the full-precision tensors gives a computed file of 49.37 MB against the 52 MB the runtime reports, a residual of 2.63 MB, which is the embedded tokenizer whose size the documentation does not state and which 32,768 pieces with merge ranks would plausibly occupy. The model of the artifact reproduces the artifact’s own size to within 5 percent without being told it.

The prediction that follows is checkable. With 48 MB resident and 48.87 MB of weights, the runtime must fetch 0.87 MB from the card per token. The documentation, written without reference to any of this, says it “streams the last few MB from the memory stick per token”.

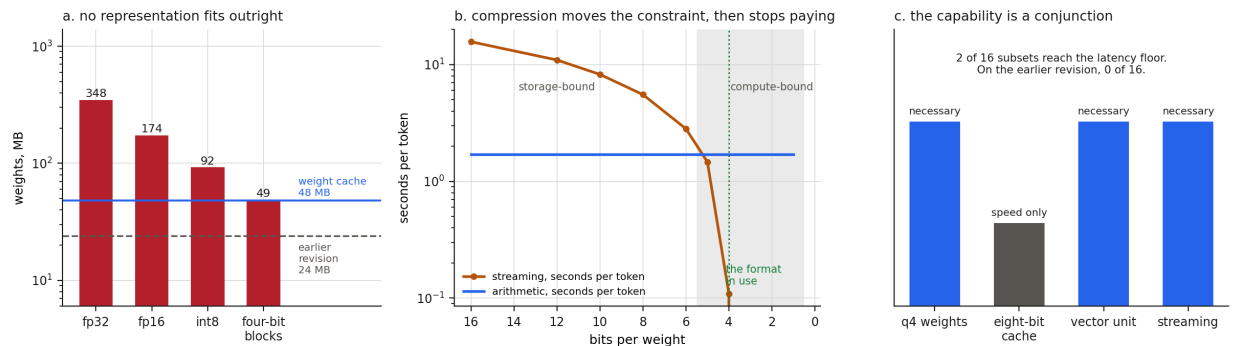


Figure 1: The technique lattice. (a) Weights in each representation against the 48 MB the console can cache and the 24 MB an earlier revision could; no representation fits outright. (b) Streaming time and arithmetic time per token as the representation narrows, with the region in which arithmetic dominates shaded and the format actually used marked. (c) Which of the four techniques appear in every configuration that reaches the latency floor.

Now take the four techniques the runtime uses, the four-bit weights, the 8-bit key-value cache, the vector-unit kernels, and the streaming, and evaluate all 16 subsets against a floor of 0.4 tokens per second. Two subsets clear it. The minimal one is four-bit weights with vector kernels and streaming, and those three are individually necessary: remove any one and no configuration runs. The 8-bit cache is not necessary. It returns 3 MB to the weight cache and shortens the stream, and at a floor of 0.4 tokens per second the system runs without it; raise the floor to 0.46 and it becomes necessary too. That is a real distinction and worth keeping: three of these techniques produce the capability and the fourth produces the experience of it. The distinction is also fragile in a stated

direction. At a card bandwidth of 4 MB per second rather than 8, only one subset clears the floor and all four techniques become necessary.

On the 32 MB revision of the same console, with the same 8 MB of runtime state, 0 of the 16 subsets clear the floor. The best any configuration reaches is 0.17 tokens per second. This is the sharpest thing in the case: the same software, the same year, the same accumulated knowledge, one hardware revision earlier, and there is no capability at all. Extensibility belongs to a machine with enough headroom for the accumulated stock to reach across the remaining gap, and the gap here was closed by the doubling of memory in the console's second revision. Programmability alone does not deliver it.

The last row of the arithmetic is the one nobody would guess. As the representation narrows, streaming time falls and arithmetic time does not, so the binding constraint migrates. At or above 6 bits per weight the system is storage-bound; at or below 5 bits it is arithmetic-bound. The format in use is 4 bits, one step past the crossing, which means the runtime has already spent all the capability that compression had to give. A perfect representation, one that occupied no space at all, would raise throughput from 0.55 to 0.585 tokens per second, a factor of 1.06. Every further gain on this device has to come from arithmetic. Compression is capability-producing up to a point that can be computed, and past it compression is decoration.

Why continuous improvement looks like discontinuous capability

The console case is one artifact. The general claim needs the distribution, and the best available census of it is not encouraging. Sherry and Thompson (2021) examined 57 textbooks and more than 1137 papers, identified 113 algorithm families with an average of eight algorithms each, and found 276 algorithms in total, an average of 1.44 improvements after the initial one in each family. Around half of the families experience little or no improvement. Of the rest, 14 percent experience transformative improvement. At a problem size of one thousand, 18 percent of families improved faster than hardware; at one million, 30 percent; at one billion, 43 percent; and the median family improved 6, 15, and 28 percent a year at those three sizes. Only at a problem size around a trillion did the median family beat hardware. The direction of travel is nonetheless the one Leiserson et al. (2020) describe, in which performance increasingly has to come from software, algorithms, and architecture because transistor scaling no longer supplies it.

The other measured series agree on direction and disagree on magnitude, which is what one would expect from a heavy-tailed distribution sampled at different places. Training a classifier to AlexNet-level ImageNet performance took 44 times fewer operations in 2019 than in 2012, an efficiency doubling every 16 months, against the 11-fold improvement Moore's law would have given over the same span (Hernandez and Brown, 2020). Compute-augmenting innovations in computer vision halve requirements about every nine months, with an interval running from 4 to 25 (Erdil and Besiroglu, 2022). For language models the compute needed to reach a fixed performance threshold halved about every 8 months, with an interval from 5 to 14 (Ho et al., 2024). In mathematical programming between 2001 and 2020, hardware improved about 20-fold while algorithms improved about 9-fold for linear programming and 50-fold for mixed-integer programming (Koch

et al., 2022).

That last paper contains the sentence this section exists to explain. Its authors note that their factors “considerably underestimate the progress made on the algorithmic side: many problem instances can nowadays be solved within seconds, which the old codes are not able to solve within any reasonable time.” A speedup ratio cannot represent a change from impossible to routine. The instrument the field uses to measure progress is blind to the specific event that makes progress feel like a change in kind, and that event is a budget crossing.

Model it. Draw 113 families from a three-component mixture whose parameters are chosen to reproduce four of the census numbers and nothing else, giving a median rate of 0.145, a transformative share of 0.124 against a design value of 0.14, and a share above hardware of 0.301. The hardware rate is not stipulated: reading it off as the seventieth percentile of the mixture gives 0.446 a year, which is inside the range benchmark series have shown, so the mixture and the census are at least coherent (Moore, 2006 [1965]; Hennessy and Patterson, 2019; Denning and Lewis, 2017). Improvements arrive as a small number of jumps per family rather than smoothly, at the census rate of 1.44 after the first, because that is what the census reports: hardware improved smoothly while algorithms “experience large, but infrequent improvements”. Give each task two resource dimensions and a budget it must satisfy in both.

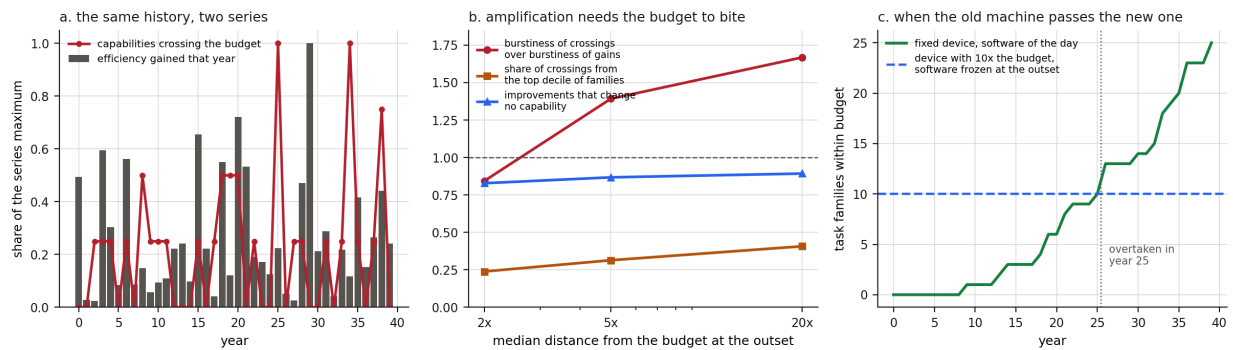


Figure 2: Threshold amplification. (a) Efficiency gained each year against capabilities crossing the budget each year, in one history at a median distance of 5 times budget. (b) Three measures against the distance from the budget at the outset: the burstiness of crossings relative to the burstiness of the gains, the concentration of crossings in the top decile of families, and the share of improvement events that change no capability. (c) The fixed device under improving software against a device with 10 times the budget under software frozen at the outset.

Most improvement does nothing visible. Across the three regimes examined, between 0.828 and 0.893 of all improvement events change no capability predicate, because the dimension they improve was slack rather than binding, or because the gain fell short of the crossing. This is the fate of the great majority of engineering effort under a conjunctive budget, and it is why the experience of using a machine can stay flat for years while its literature fills with results.

Amplification is real and it is conditional. The coefficient of variation of annual capability crossings, divided by the same statistic for the annual efficiency gains that cause them, is 0.84 when tasks

start at 2 times their budget, 1.39 at 5 times, and 1.67 at 20 times. At 2 times the budget there is no amplification at all: the crossings are smoother than the gains. The threshold effect requires the threshold to be far away. Concentration behaves the same way, with the top decile of families contributing 0.24, 0.31, and 0.41 of all crossings across the three regimes, and the share of families that never cross in forty years rising from 0.51 to 0.69 to 0.76. The demanding regime is the one where a small number of task families deliver most of what looks like a change in the character of the machine.

The comparison that follows from this is the one a purchasing decision actually turns on. Put the fixed device with the software of the day against a device with 10 times its budget running the software of the earlier date. In the two easier regimes the newer device is never overtaken, because a tenfold budget covers a great many tasks that were nearly in reach. In the demanding regime it is overtaken in year 25. Old hardware with new software beats new hardware with old software only on the problems that were far out of reach for both, which is a narrow claim and the only one the model supports.

Three constraints keep this from becoming a general thesis about software. No algorithm can be better than another averaged over all problems (Wolpert and Macready, 1997), so every claim here is indexed to a task family and a distribution. Information processing has a physical floor (Landauer, 1961). And the causal arrow runs the other way as well: which algorithms get discovered and refined is shaped by which hardware happens to exist, so the hardware-software separation used throughout this paper is an experimental control rather than a fact about the world (Hooker, 2021). The console case is a small instance of exactly that, since the model it runs exists because a quite different fleet of machines was available to train it.

The frontier that grows and the frontier you can run

Everything so far argues that the reachable set expands. The last result argues that the set you can actually reach shrinks, for a reason the first two sections do not contain.

Knowledge accumulates monotonically. Deployment does not, because deployment is a conjunction over a stack. The console demonstration requires custom firmware on the device, a cross-compilation toolchain, the runtime source, the quantised weight file, and the tokenizer, and it stops working if any one of them becomes unobtainable. Rothenberg (1995) made the general point three decades ago, that preserving the bits is not preserving the ability to use them, and the threat models developed since are about exactly this kind of compound dependency (Rosenthal et al., 2005). Emulation is a partial substitute for a surviving toolchain, and one whose fidelity is itself measurable rather than assumed (Guttenbrunner and Rauber, 2012).

Give each component an annual hazard of becoming unrunnable and a chance of being restored by someone re-reporting it. A component alive with probability a makes a stack of depth d deployable with probability a^d , so the deployable stock converges to a fraction of the known stock rather than tracking it. At a hazard of 0.03 a year against a revival rate of 0.12, a single component is available 0.80 of the time, and the stationary deployable fraction falls to 0.64, 0.512, 0.328, 0.168, and 0.069

at depths of 2, 3, 5, 8, and 12. Half the known stock is unrunnable by depth 4. Simulating arrivals and failures rather than taking the stationary limit gives deployable shares at forty years of 0.83, 0.70, 0.58, 0.43, 0.25, and 0.16 across the same depths. A five-deep stack, which is what the console demonstration is, sits at 0.43.

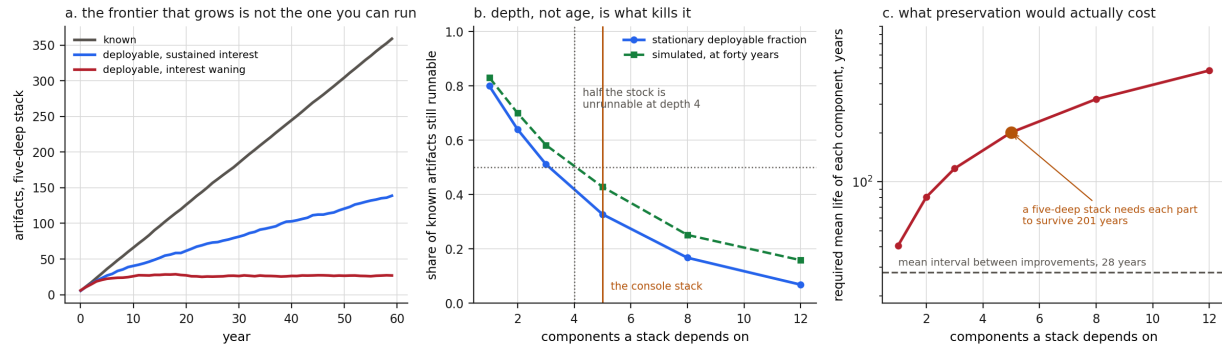


Figure 3: The deployable frontier. (a) Known and deployable artifacts for a five-component stack, under sustained interest and under waning interest. (b) The share of known artifacts still runnable against the depth of the stack, in the stationary limit and in simulation at forty years. (c) The mean component life a stack of each depth would need for its half-life to match the interval between improvements in its field.

Sustained arrivals hide this, because a growing stock of new artifacts masks the decay of the old. Withdraw the arrivals, which is what happens to every platform eventually, and the frontier turns over. With interest decaying on a twelve-year scale, a five-deep stack peaks in year 18 and a twelve-deep stack peaks in year 6 and falls to 0.39 of its peak by year 60. Nothing was forgotten in the model. The knowledge is all still there. It simply cannot be run.

The number that states the cost of preventing this is the one to keep. Set a stack's half-life equal to the mean interval between improvements in its own field, which the census puts at 27.8 years given 1.44 improvements per family over four decades. A single component would have to survive a mean of 41 years to meet that bar. A five-component stack would have to have each of its parts survive 201 years. Preservation of a computational capability is not a matter of keeping a machine in a cupboard, and it is roughly two orders of magnitude more demanding than the intuition that says a working device is the hard part to find.

What this is not

The thesis is easy to overstate, and the overstatements are worth naming because each has a specific answer.

It is not a claim that software gives hardware new physical powers. The set of physically reachable configurations is fixed; what moves is the set anyone knows how to reach within a budget.

It is not Turing universality restated. Universality says the console could in principle simulate anything given unbounded time and tape. It is silent on 48 MB, on two seconds, and on whether the

output is worth reading, and the entire content of the console case lives in those three constraints.

It is not a claim that the console became intelligent. The runtime’s own author says the model is “not really useful for anything”, and the case is offered as a demonstration of a threshold crossing rather than of a quality.

It is not a general law about software progress. The census says half of all algorithm families barely improve at all, the model here is calibrated to reproduce that, and in the model between 0.51 and 0.76 of families never cross a budget in forty years depending on how far the budget sits.

It is not a claim about self-contained development. The parameters were trained elsewhere on machines the console could not emulate in a human lifetime, and the honest accounting separates the cost of deploying a capability from the cost of discovering it. The console demonstrates deployment extensibility. Amortised over enough deployments the development cost per use falls, which is a different claim requiring a different measurement.

It is not an argument that old hardware is environmentally preferable. Energy per completed task may remain poor, and nothing here compares lifecycles. The narrower conclusion the arithmetic supports is that obsolescence cannot be inferred from a device’s current software-defined function.

And it is not a clean separation of hardware from software. The separation is a control, and Hooker’s argument about which algorithms win because of which hardware exists is the reason it is only a control.

What would break this

The first result is arithmetic and can be checked directly. If a careful measurement on the hardware shows the runtime streaming far more or far less than 0.87 MB per token, or shows the system storage-bound rather than arithmetic-bound at four-bit weights, the model of the artifact is wrong in a way that no reinterpretation rescues. The ablation is also runnable: build the seven configurations the lattice says should fail and confirm that they do, and build the two it says should run and confirm the throughput. Any configuration that runs and should not falsifies the necessity claim. The prediction that a PSP-1000 cannot be made to work with any subset of these techniques is the strongest single falsifier available, and the cheapest.

The second result depends on a mixture calibrated to a census of exact algorithms with exact solutions, which is a deliberately conservative sample, since approximate and heuristic methods are excluded and those are where quantisation and distillation live. A census extended to approximate methods might show a different tail, and the model’s calibration should move with it. The specific prediction that would break the amplification result is a domain in which capability crossings are measured to be smoother than the efficiency gains that produce them while the tasks are far from budget. The prediction that would break its conditionality is a domain where amplification appears even though the tasks were nearly feasible to begin with.

The third result rests on a hazard and a revival rate that nobody has measured. They could be measured. Take a corpus of released artifacts of known dependency depth, attempt to build and

run each at intervals, and estimate the hazard and the revival rate directly. If the hazard for well-maintained ecosystems is far below the 0.0246 a year that a single component would need, the decay result softens to a claim about neglected platforms rather than about software generally. If it is above, the deployable frontier is falling in places nobody is measuring, and the field is discovering capabilities faster than it is keeping them.

The claim that survives all three is modest and, this paper argues, correct. A statement of the form *this machine can do that* is a fact about a machine, an artifact, a task, a budget, a quality threshold, and a date. Dropping the last four terms is what makes the eventual result feel unreasonable.

References

- Aaronson, S. (2013). Why philosophers should care about computational complexity. In B. J. Copeland, C. J. Posy, and O. Shagrir (eds.), *Computability: Turing, Gödel, Church, and Beyond*, 261–328. MIT Press.
- Blum, M. (1967). A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2), 322–336.
- Denning, P. J., and Lewis, T. G. (2017). Exponential laws of computing growth. *Communications of the ACM*, 60(1), 54–65.
- Erdil, E., and Besiroglu, T. (2022). Algorithmic progress in computer vision. arXiv:2212.05153.
- Frantar, E., and Alistarh, D. (2022). Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*, 35, 4475–4488.
- Gu, A., and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752.
- Guttenbrunner, M., and Rauber, A. (2012). A measurement framework for evaluating emulators for digital preservation. *ACM Transactions on Information Systems*, 30(2), 1–28.
- Halevy, A., Norvig, P., and Pereira, F. (2009). The unreasonable effectiveness of data. *IEEE Intelligent Systems*, 24(2), 8–12.
- Hartmanis, J., and Stearns, R. E. (1965). On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117, 285–306.
- Hennessy, J. L., and Patterson, D. A. (2019). A new golden age for computer architecture. *Communications of the ACM*, 62(2), 48–60.
- Hernandez, D., and Brown, T. B. (2020). Measuring the algorithmic efficiency of neural networks. arXiv:2005.04305.
- Ho, A., Besiroglu, T., Erdil, E., Owen, D., Rahman, R., Guo, Z. C., Atkinson, D., Thompson, N., and Sevilla, J. (2024). Algorithmic progress in language models. arXiv:2403.05812.

- Hooker, S. (2021). The hardware lottery. *Communications of the ACM*, 64(12), 58–65.
- Hutter, M. (2002). The fastest and shortest algorithm for all well-defined problems. *International Journal of Foundations of Computer Science*, 13(3), 431–443.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–2713.
- Koch, T., Berthold, T., Pedersen, J., and Vanaret, C. (2022). Progress in mathematical programming solvers from 2001 to 2020. *EURO Journal on Computational Optimization*, 10, 100031.
- Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3), 183–191.
- Leiserson, C. E., Thompson, N. C., Emer, J. S., Kuszmaul, B. C., Lampson, B. W., Sanchez, D., and Schardl, T. B. (2020). There’s plenty of room at the Top: What will drive computer performance after Moore’s law? *Science*, 368(6495), eaam9744.
- Li, M., and Vitányi, P. (2019). *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed. Springer.
- Massalin, H. (1987). Superoptimizer: A look at the smallest program. *ACM SIGARCH Computer Architecture News*, 15(5), 122–126.
- Moor, J. H. (1985). What is computer ethics? *Metaphilosophy*, 16(4), 266–275.
- Moore, G. E. (2006 [1965]). Cramming more components onto integrated circuits. *IEEE Solid-State Circuits Society Newsletter*, 11(3), 33–35.
- Piccinini, G. (2007). Computing mechanisms. *Philosophy of Science*, 74(4), 501–526.
- Piccinini, G. (2008). Computers. *Pacific Philosophical Quarterly*, 89(1), 32–73.
- Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2), 358–366.
- Rosenthal, D. S. H., Robertson, T., Lipkis, T., Reich, V., and Morabito, S. (2005). Requirements for digital preservation systems: A bottom-up approach. *D-Lib Magazine*, 11(11).
- Rothenberg, J. (1995). Ensuring the longevity of digital documents. *Scientific American*, 272(1), 42–47.
- Sherry, Y., and Thompson, N. C. (2021). How fast do algorithms improve? *Proceedings of the IEEE*, 109(11), 1768–1777.
- Technology Innovation Institute (2026). Falcon-H1-Tiny-90M-Instruct model card.
- Thatblend (2026). LLMPSP: A custom LLM runtime for the Sony PSP.
- Turing, A. M. (1937). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265.

Wigner, E. P. (1960). The unreasonable effectiveness of mathematics in the natural sciences. *Communications on Pure and Applied Mathematics*, 13(1), 1–14.

Wolpert, D. H., and Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82.